실제 예제와 사용법

개요

이 문서는 Plonky3 라이브러리의 실제 사용법을 구체적인 예제를 통해 설명합니다. examples 디렉토리에 포함된 증명 생성 예제들을 단계별로 분석하고, 벤치마크 실행 방법과 결과 해석, 그리고 커스텀 AIR 구현 가이드를 제공합니다.

Plonky3는 다양한 해시 함수(Blake3, Keccak, Poseidon2)와 유한체(BabyBear, KoalaBear, Mersenne31)의 조합으로 STARK 증명을 생성할 수 있는 유연한 프레임워크입니다. 이 문서를 통해 실제 프로덕션 환경에서 어떻게 활용할 수 있는지 이해할 수 있습니다.

프로젝트 구조 이해

Examples 디렉토리 구조

```
examples/
├── Cargo.toml # 의존성 및 빌드 설정
├── examples/
| └── prove_prime_field_31.rs # 메인 예제 실행 파일
└── src/
├── lib.rs # 라이브러리 진입점
├── airs.rs # AIR 추상화 및 구현
├── proofs.rs # 증명 생성 함수들
├── parsers.rs # CLI 파라미터 파싱
```

```
├── dfts.rs # DFT 선택 옵션

├── types.rs # 타입 정의

└── tests.rs # 통합 테스트
```

핵심 컴포넌트

1. **ProofObjective**: 증명하고자 하는 해시 함수를 추상화한 enum
2. **증명 함수들**: 각 필드와 해시 함수 조합별 증명 생성 함수
3. **설정 타입들**: 각 조합에 맞는 STARK 설정 타입들
4. **CLI 인터페이스**: 사용자 친화적인 명령줄 인터페이스

메인 예제 분석: prove_prime_field_31.rs

1. 기본 구조와 의존성

메인 예제는 clap을 사용한 CLI 애플리케이션으로 구성되어 있습니다:

```
use clap::Parser;

use p3_examples::airs::ProofObjective;

use p3_examples::dfts::DftChoice;

use p3_examples::parsers::{DftOptions, FieldOptions, MerkleHashOptions, ProofOpt

use p3_examples::proofs::{

prove_m31_keccak, prove_m31_poseidon2, prove_monty31_keccak, prove_monty31_posei

report_result,

};
```

2. 명령줄 인터페이스

```
#[derive(Parser, Debug)]
struct Args {
/// 증명에 사용할 유한체
#[arg(short, long, ignore_case = true, value_enum)]
field: FieldOptions,

/// 증명하고자 하는 대상
#[arg(short, long, ignore_case = true, value_enum)]
objective: ProofOptions,

/// trace 길이의 log2 값
#[arg(short, long)]
log_trace_length: u8,

/// 사용할 이산 푸리에 변환
#[arg(short, long, ignore_case = true, value_enum, default_value_t = DftOptions:
discrete_fourier_transform: DftOptions,

/// 머클 트리 구성에 사용할 해시 함수
#[arg(short, long, ignore_case = true, value_enum)]
merkle_hash: MerkleHashOptions,
```

```
}
```

3. 실행 예제들

Blake3 증명 생성 (BabyBear 필드, Poseidon2 머클 트리)

```
cargo run --example prove_prime_field_31 -- \
--field baby-bear \
--objective blake-3-permutations \
--log-trace-length 10 \
--discrete-fourier-transform radix-2-dit-parallel \
--merkle-hash poseidon-2
```

이 명령은 다음을 수행합니다:

- 2^10 = 1024개의 Blake3 순열 증명
- BabyBear 필드 사용
- Radix-2 DIT 병렬 FFT 사용
- Poseidon2 해시로 머클 트리 구성

Keccak 증명 생성 (Mersenne31 필드, Keccak 머클 트리)

```
cargo run --example prove_prime_field_31 -- \
--field mersenne-31 \
--objective keccak-f-permutations \
--log-trace-length 8 \
```

```
--merkle-hash keccak-f
```

Mersenne31 필드는 Circle STARK를 사용하므로 DFT 옵션이 필요하지 않습니다.

Poseidon2 증명 생성 (KoalaBear 필드)

```
cargo run --example prove_prime_field_31 -- \
--field koala-bear \
--objective poseidon-2-permutations \
--log-trace-length 12 \
--discrete-fourier-transform recursive-dft \
--merkle-hash poseidon-2
```

증명 생성 과정 상세 분석

1. 필드별 설정

각 유한체는 고유한 특성과 최적화를 가집니다:

BabyBear (2^31 - 2^27 + 1)

```rust
type EF = BinomialExtensionField<BabyBear, 4>;
```

```rust
// Poseidon2 AIR 설정
const SBOX_DEGREE: u64 = 7;
const SBOX_REGISTERS: usize = 1;
```

```
const PARTIAL_ROUNDS: usize = 13;
```

KoalaBear (2^31 - 2^24 + 1)

```
type EF = BinomialExtensionField<KoalaBear, 4>;

// Poseidon2 AIR 설정
const SBOX_DEGREE: u64 = 3;
const SBOX_REGISTERS: usize = 0;
const PARTIAL_ROUNDS: usize = 20;
```

Mersenne31 (2^31 - 1)

```
type EF = BinomialExtensionField<Mersenne31, 3>;

// Poseidon2 AIR 설정
const SBOX_DEGREE: u64 = 5;
const SBOX_REGISTERS: usize = 1;
const PARTIAL_ROUNDS: usize = 14;
```

2. AIR 구성

각 해시 함수는 고유한 AIR 구현을 가집니다:

```
let  proof_goal = match  args.objective {
```

```
ProofOptions::Blake3Permutations => ProofObjective::Blake3(Blake3Air {}),

ProofOptions::KeccakFPermutations => ProofObjective::Keccak(KeccakAir {}),

ProofOptions::Poseidon2Permutations => {

let constants = RoundConstants::from_rng(&mut rng);

let p2_air = VectorizedPoseidon2Air::new(constants);

ProofObjective::Poseidon2(p2_air)

}

};
```

3. DFT 선택

DFT는 다항식 연산의 핵심입니다:

```
let dft = match args.discrete_fourier_transform {

DftOptions::RecursiveDft => {

DftChoice::Recursive(RecursiveDft::new(trace_height << 1))

}

DftOptions::Radix2DitParallel => DftChoice::Parallel(Radix2DitParallel::default(

DftOptions::None => panic!("DFT 옵션을 지정해주세요"),

};
```

4. 증명 생성 및 검증

```
match args.merkle_hash {
```

```
MerkleHashOptions::KeccakF => {
let result = prove_monty31_keccak::<_, EF, _, _>(proof_goal, dft, num_hashes);
report_result(result);
}
MerkleHashOptions::Poseidon2 => {
let perm16 = Poseidon2BabyBear::<16>::new_from_rng_128(&mut rng);
let perm24 = Poseidon2BabyBear::<24>::new_from_rng_128(&mut rng);
let result = prove_monty31_poseidon2::<_, EF, _, _, _, _>(
proof_goal, dft, num_hashes, perm16, perm24,
);
report_result(result);
}
};
```

증명 함수 구현 분석

1. prove_monty31_keccak 함수

```
pub fn prove_monty31_keccak<
F: PrimeField32 + TwoAdicField,
EF: ExtensionField<F>,
DFT: TwoAdicSubgroupDft<F>,
PG: ExampleHashAir<F, KeccakStarkConfig<F, EF, DFT>>,
```

```
>(
proof_goal: PG,
dft: DFT,
num_hashes: usize,
) -> Result<(), impl Debug>
```

이 함수는 다음 단계를 수행합니다:

1. **MMCS 설정**: Keccak 기반 머클 트리 구성

```
let val_mmcs = get_keccak_mmcs();
let challenge_mmcs = ExtensionMmcs::<F, EF, _>::new(val_mmcs.clone());
```

2. **FRI 매개변수 생성**: 벤치마크용 최적화된 매개변수

```
let fri_params = create_benchmark_fri_params(challenge_mmcs);
```

3. **Trace 생성**: AIR에서 실제 계산 trace 생성

```
let trace = proof_goal.generate_trace_rows(num_hashes, fri_params.log_blowup);
```

4. **PCS 및 Challenger 설정**:

```
let pcs = TwoAdicFriPcs::new(dft, val_mmcs, fri_params);
let challenger = SerializingChallenger32::from_hasher(vec![], Keccak256Hash {})
```

5. **증명 생성 및 검증**:

```
let config = KeccakStarkConfig::new(pcs, challenger);
let proof = prove(&config, &proof_goal, trace, &vec![]);
verify(&config, &proof_goal, &proof, &vec![])
```

2. prove_m31_keccak 함수 (Circle STARK)

Mersenne31 필드는 Circle STARK를 사용합니다:

```
pub fn prove_m31_keccak<
PG: ExampleHashAir<
Mersenne31,
KeccakCircleStarkConfig<Mersenne31, BinomialExtensionField<Mersenne31, 3>>,
>,
>(
proof_goal: PG,
num_hashes: usize,
) -> Result<(), impl Debug>
```

주요 차이점:

- `CirclePcs` 사용 (FFT 대신)
- 3차 확장 필드 사용
- DFT 불필요

벤치마크 실행 및 분석

1. 벤치마크 실행 방법

필드별 벤치마크

```
# BabyBear 필드 벤치마크
cargo bench --package p3-baby-bear

# KoalaBear 필드 벤치마크
cargo bench --package p3-koala-bear

# Mersenne31 필드 벤치마크
cargo bench --package p3-mersenne-31
```

해시 함수별 벤치마크

```
# Poseidon2 벤치마크
cargo bench --package p3-poseidon2

# Keccak 벤치마크
cargo bench --package p3-keccak

# Blake3 벤치마크 (AIR 포함)
cargo bench --package p3-blake3-air
```

DFT/FFT 벤치마크

```
# DFT 구현 벤치마크
cargo bench --package p3-dft

# Circle FFT 벤치마크
cargo bench --package p3-circle
```

2. 벤치마크 결과 해석

필드 연산 성능

벤치마크는 다음 메트릭을 측정합니다:

1. **기본 연산 지연시간 (Latency)**:
   - 단일 연산의 실행 시간
   - CPU 파이프라인 최적화 효과 측정
2. **처리량 (Throughput)**:
   - 단위 시간당 처리 가능한 연산 수
   - SIMD 최적화 효과 측정
3. **벡터 내적 (Dot Product)**:
   - 다양한 크기의 벡터 연산 성능
   - 메모리 접근 패턴 최적화 효과

예시 결과 분석

```
BabyBear/add_latency time: [1.2345 ns 1.2456 ns 1.2567 ns]
BabyBear/add_throughput time: [0.1234 ns 0.1245 ns 0.1256 ns]
```

```
BabyBear/mul_latency time: [2.3456 ns 2.3567 ns 2.3678 ns]

BabyBear/mul_throughput time: [0.2345 ns 0.2356 ns 0.2367 ns]
```

해석:

- 덧셈 지연시간: ~1.25ns (단일 연산)
- 덧셈 처리량: ~0.125ns (SIMD로 8개 병렬 처리 시)
- 곱셈이 덧셈보다 약 2배 느림
- SIMD 최적화로 약 10배 성능 향상

3. 성능 최적화 포인트

SIMD 최적화 효과

```
// 스칼라 연산
fn scalar_add(a: BabyBear, b: BabyBear) -> BabyBear {
a + b  // ~1.25ns
}


// SIMD 연산 (AVX2)
fn simd_add(a: [BabyBear; 8], b: [BabyBear; 8]) -> [BabyBear; 8] {
// 8개 동시 처리, 총 ~1.0ns (개당 0.125ns)
a.zip(b).map(|(x, y)| x + y)
}
```

메모리 접근 패턴

```
// 효율적인 패턴: 순차 접근
for i in 0..n {
result[i] = data[i] * coefficient;
}

// 비효율적인 패턴: 랜덤 접근
for i in 0..n {
result[i] = data[random_indices[i]] * coefficient;
}
```

통합 테스트 분석

1. 테스트 구조

`examples/src/tests.rs` 는 다양한 조합의 end-to-end 테스트를 제공합니다:

```
#[test]
fn test_end_to_end_koalabear_vectorized_poseidon2_hashes_recursive_dft_poseidon
-> Result<(), impl Debug> {
// 설정
type EF = BinomialExtensionField<KoalaBear, 4>;
let constants = RoundConstants::from_rng(&mut rng);
// AIR 구성
let proof_goal: VectorizedPoseidon2Air<...> = VectorizedPoseidon2Air::new(const
```

```
// DFT 선택
let  dft = DftChoice::Recursive(RecursiveDft::new(TRACE_SIZE >> VECTOR_LEN));
// 증명 생성 및 검증
prove_monty31_poseidon2::<_, EF, _, _, _, _>(proof_goal, dft, TRACE_SIZE, perm16
}
```

2. 테스트 매트릭스

| 필드 | 해시 함수 | DFT | 머클 해시 | 테스트 함수 |

|------|----------|-----|----------|----------|

| KoalaBear | Poseidon2 (벡터화) | Recursive | Poseidon2 | ✅ |

| KoalaBear | Poseidon2 | Recursive | Keccak | ✅ |

| KoalaBear | Keccak | Parallel | Keccak | ✅ |

| BabyBear | Poseidon2 (벡터화) | Recursive | Poseidon2 | ✅ |

| BabyBear | Poseidon2 | Parallel | Poseidon2 | ✅ |

| BabyBear | Blake3 | Parallel | Poseidon2 | ✅ |

| Mersenne31 | Keccak | Circle | Keccak | ✅ |

| Mersenne31 | Blake3 | Circle | Keccak | ✅ |

| Mersenne31 | Poseidon2 (벡터화) | Circle | Poseidon2 | ✅ |

| Mersenne31 | Poseidon2 | Circle | Keccak | ✅ |

3. 테스트 실행

```
# 모든 통합 테스트 실행
```

```
cargo test --package p3-examples

# 특정 테스트 실행
cargo test --package p3-examples test_end_to_end_koalabear_vectorized_poseid

# 릴리스 모드로 테스트 (성능 최적화 적용)
cargo test --package p3-examples --release
```

커스텀 AIR 구현 가이드

1. 기본 AIR 인터페이스

새로운 해시 함수나 계산을 위한 AIR을 구현하려면 다음 트레이트들을 구현해야 합니다:

```
use p3_air::{Air, AirBuilder, BaseAir};
use p3_matrix::dense::RowMajorMatrix;

pub struct CustomAir {
// AIR 설정 매개변수들
}

impl<F: Field> BaseAir<F> for CustomAir {
fn width(&self) -> usize {
// trace의 열 개수 반환
```

```
16
}
}

impl<AB: AirBuilder> Air<AB> for CustomAir {
fn eval(&self, builder: &mut AB) {
// 제약조건 정의
let main = builder.main();
let local = main.row_slice(0);
let next = main.row_slice(1);
// 예: 상태 전이 제약조건
for i in 0..self.width() {
builder.when_transition().assert_eq(
next[i],
local[i] + AB::Expr::one()
);
}
}
}
```

2. ExampleHashAir 트레이트 구현

벤치마크와 테스트에서 사용하려면 `ExampleHashAir` 도 구현해야 합니다:

```
impl<F, SC> ExampleHashAir<F, SC> for CustomAir
where
F: PrimeField64,
SC: StarkGenericConfig,
StandardUniform: Distribution<F>,
{
fn generate_trace_rows(
&self,
num_hashes: usize,
extra_capacity_bits: usize,
) -> RowMajorMatrix<F> {
let trace_height = num_hashes.next_power_of_two() << extra_capacity_bits;
let mut trace = RowMajorMatrix::new(
vec![F::zero(); trace_height * self.width()],
self.width(),
);
// trace 데이터 생성 로직
for i in 0..num_hashes {
// 실제 해시 계산 수행
let hash_input = generate_random_input();
let hash_output = custom_hash_function(hash_input);
// trace에 기록
```

```
for j in 0..self.width() {
trace.set(i, j, hash_output[j]);
}
}
trace
}
}
```

3. 커스텀 해시 함수 예제

간단한 피보나치 수열 AIR 구현:

```
pub struct FibonacciAir {
pub num_steps: usize,
}

impl<F: Field> BaseAir<F> for FibonacciAir {
fn width(&self) -> usize {
2 // a, b 두 개의 상태
}
}

impl<AB: AirBuilder> Air<AB> for FibonacciAir {
fn eval(&self, builder: &mut AB) {
```

```
let main = builder.main();

let local = main.row_slice(0);

let next = main.row_slice(1);

// 피보나치 제약조건: next_a = local_b, next_b = local_a + local_b

builder.when_transition().assert_eq(next[0], local[1]);

builder.when_transition().assert_eq(next[1], local[0] + local[1]);

// 초기값 제약조건

builder.when_first_row().assert_eq(local[0], AB::Expr::one());

builder.when_first_row().assert_eq(local[1], AB::Expr::one());

}

}


impl<F, SC> ExampleHashAir<F, SC> for FibonacciAir

where

F: PrimeField64,

SC: StarkGenericConfig,

StandardUniform: Distribution<F>,

{

fn generate_trace_rows(

&self,

num_steps: usize,

extra_capacity_bits: usize,

) -> RowMajorMatrix<F> {
```

```
let trace_height = num_steps.next_power_of_two() << extra_capacity_bits;

let mut trace = RowMajorMatrix::new(

vec![F::zero(); trace_height * 2],

2,

);

// 피보나치 수열 생성

trace.set(0, 0, F::one()); // a = 1

trace.set(0, 1, F::one()); // b = 1

for i in 1..num_steps {

let prev_a = trace.get(i - 1, 0);

let prev_b = trace.get(i - 1, 1);

trace.set(i, 0, prev_b); // a = prev_b

trace.set(i, 1, prev_a + prev_b); // b = prev_a + prev_b

}

trace

}

}
```

4. 커스텀 AIR 사용

```
fn prove_fibonacci<F, EF, DFT>(

num_steps: usize,

dft: DFT,
```

```
) -> Result<(), impl Debug>

where

F: PrimeField32 + TwoAdicField,

EF: ExtensionField<F>,

DFT: TwoAdicSubgroupDft<F>,

StandardUniform: Distribution<F>,

{

let  fibonacci_air = FibonacciAir { num_steps };

// 기존 증명 함수와 동일한 패턴

let  val_mmcs = get_keccak_mmcs();

let  challenge_mmcs = ExtensionMmcs::<F, EF, _>::new(val_mmcs.clone());

let  fri_params = create_benchmark_fri_params(challenge_mmcs);

let  trace = fibonacci_air.generate_trace_rows(num_steps, fri_params.log_blowup)

let  pcs = TwoAdicFriPcs::new(dft, val_mmcs, fri_params);

let  challenger = SerializingChallenger32::from_hasher(vec![], Keccak256Hash {})

let  config = KeccakStarkConfig::new(pcs, challenger);

let  proof = prove(&config, &fibonacci_air, trace, &vec![]);

verify(&config, &fibonacci_air, &proof, &vec![])

}
```

성능 튜닝 가이드

1. 필드 선택 기준

| 필드 | 장점 | 단점 | 사용 사례 |

|------|------|------|----------|

| BabyBear | 빠른 곱셈, 좋은 SIMD 지원 | 작은 소수 | 일반적인 용도 |

| KoalaBear | 균형잡힌 성능 | - | 범용 애플리케이션 |

| Mersenne31 | Circle STARK, 빠른 연산 | 제한된 DFT 옵션 | 특수 최적화 |

| Goldilocks | 64비트 최적화 | 메모리 사용량 증가 | 고성능 요구사항 |

2. DFT 선택 기준

```
// 작은 trace: Recursive DFT
if trace_size < 1024 {
DftChoice::Recursive(RecursiveDft::new(trace_size))
} else {
// 큰 trace: Parallel DFT
DftChoice::Parallel(Radix2DitParallel::default())
}
```

3. 메모리 최적화

```
// 메모리 효율적인 trace 생성
fn generate_trace_streaming<F: Field>(
air: &impl Air<F>,
num_rows: usize,
) -> impl Iterator<Item = Vec<F>> {
```

```
(0..num_rows).map(move |i| {
// 필요한 시점에만 행 생성
generate_row(air, i)
})
}
```

4. 병렬 처리 최적화

```
use p3_maybe_rayon::prelude::*;

// 조건부 병렬 처리
fn  parallel_trace_generation<F: Field + Send + Sync>(
chunks: &[TraceChunk],
) -> Vec<Vec<F>> {
chunks
.maybe_par_iter() // 병렬 처리 가능 시 자동 활성화
.map(|chunk| process_chunk(chunk))
.collect()
}
```

프로덕션 사용 고려사항

1. 보안 고려사항

```
// ❌ 예제용 - 프로덕션에서 사용 금지
let mut rng = SmallRng::seed_from_u64(1);


// ✅ 프로덕션용 - 암호학적으로 안전한 난수 생성기
use rand::rngs::OsRng;
let mut rng = OsRng;
```

2. 에러 처리

```
pub fn prove_with_error_handling<F, EF, DFT, PG>(
proof_goal: PG,
dft: DFT,
num_hashes: usize,
) -> Result<Vec<u8>, ProofError>
where
// ... 타입 제약조건들
{
let trace = proof_goal
.generate_trace_rows(num_hashes, fri_params.log_blowup)
.map_err(ProofError::TraceGeneration)?;
let proof = prove(&config, &proof_goal, trace, &vec![])
.map_err(ProofError::ProofGeneration)?;
let verification = verify(&config, &proof_goal, &proof, &vec![])
```

```
.map_err(ProofError::Verification)?;

// 증명 직렬화

let proof_bytes = bincode::serde::encode_to_vec(&proof, config)

.map_err(ProofError::Serialization)?;

Ok(proof_bytes)

}


#[derive(Debug, thiserror::Error)]

pub enum ProofError {

#[error("Trace generation failed: {0}")]

TraceGeneration(String),

#[error("Proof generation failed: {0}")]

ProofGeneration(String),

#[error("Verification failed: {0}")]

Verification(String),

#[error("Serialization failed: {0}")]

Serialization(String),

}
```

3. 설정 관리

```
#[derive(Debug, Clone, serde::Serialize, serde::Deserialize)]

pub struct ProofConfig {
```

```
pub field_type: FieldType,
pub hash_function: HashFunction,
pub merkle_hash: MerkleHashType,
pub dft_type: DftType,
pub security_level: u8,
pub blowup_factor: usize,
}

impl ProofConfig {
pub fn production_default() -> Self {
Self {
field_type: FieldType::BabyBear,
hash_function: HashFunction::Poseidon2,
merkle_hash: MerkleHashType::Poseidon2,
dft_type: DftType::Radix2DitParallel,
security_level: 128,
blowup_factor: 8,
}
}
pub fn validate(&self) -> Result<(), ConfigError> {
if self.security_level < 80 {
return Err(ConfigError::InsufficientSecurity);
}
```

```
if !self.field_type.supports_dft(&self.dft_type) {
return Err(ConfigError::IncompatibleDft);
}
Ok(())
}
}
```

결론

Plonky3의 examples는 실제 STARK 증명 시스템을 구축하고 최적화하는 방법을 보여주는 포괄적인 가이드입니다. 다양한 필드, 해시 함수, DFT 구현의 조합을 통해 특정 사용 사례에 최적화된 증명 시스템을 구축할 수 있습니다.

핵심 포인트:

1. **모듈성**: 각 컴포넌트를 독립적으로 선택하고 조합 가능
2. **성능**: SIMD 최적화와 병렬 처리를 통한 고성능 구현
3. **유연성**: 커스텀 AIR 구현을 통한 확장 가능성
4. **실용성**: 프로덕션 환경에서 사용 가능한 완전한 구현

이러한 예제들을 통해 Plonky3의 강력함과 유연성을 이해하고, 실제 애플리케이션에 적용할 수 있는 기반을 마련할 수 있습니다.